Bitácora de prompts · EcoTrack EmissionsTracker

Proyecto Integrador «El Ciclo Spec-to-Code en Acción», módulo 3, «El Núcleo UI (React + IA)». Stack: Vite 8, React 19, TypeScript 6 en modo estricto, Tailwind CSS 4, Supabase JS 2 y Vitest 5, todo con pnpm.

Esta bitácora recoge los cuatro prompts clave y cómo cambió el código de uno a otro. Ninguna versión se reconstruyó al final: cada respuesta de la IA se guardó tal cual en `bitacora/vN/` y se verificó con las mismas herramientas antes de escribir el siguiente prompt. Cada prompt parte de esas salidas reales (errores de `tsc`, avisos de `oxlint`, pruebas que fallan y pruebas de RLS), no de una idea general de «mejorar el código».

Paso	Prompt	Versión	tsc	oxlint	Pruebas	RLS (anon)	Bundle JS con variables
1	Estructura	`v1`	0 errores	0	sin pruebas	sin SQL	226,15 kB (no usa Supabase)
2	Lógica del hook	`v2`	0 errores	2 avisos	**2 de 4 fallan**	**4 de 9**	444,55 kB
3	Refinamiento	`v3`	0 errores (modo más estricto)	**1 error, 2 avisos**	27 de 27	sin cambios	448,34 kB
4	Auditoría	`v4`	0 errores	**0**	37 de 37	**9 de 9**	449,13 kB
Final	Ajuste tras las capturas	`src/`	0 errores	0	37 de 37	9 de 9	449,37 kB (128,33 kB gzip)

Fuentes: `bitacora/v1/verificacion.txt` a `v4/verificacion.txt`, `v2/rls-pruebas.txt`, `v4/rls-pruebas.txt` y `docs/evidencia/verificacion-final.txt`.

---

1. Prompt de estructura

Qué se buscaba: que la IA propusiera la arquitectura a partir de la especificación, sin escribir todavía la lógica. Se le dio el stack exacto (versiones, `verbatimModuleSyntax` y Tailwind 4 sin archivo de configuración) y una regla de dependencias: la UI nunca importa Supabase.

```
Actúa como arquitecto frontend senior. Vamos a construir el núcleo de la
interfaz del
"Dashboard de Emisiones en Tiempo Real" de EcoTrack […]
Stack fijo (ya instalado, no propongas otro): Vite 8 + React 19 +
TypeScript 6 con "strict": true
y "verbatimModuleSyntax": true; Tailwind CSS 4 con @tailwindcss/vite;
@supabase/supabase-js 2.
[…]
En ESTE paso solo quiero la arquitectura, no la lógica de datos:
1. Propón el árbol de archivos de src/ y la responsabilidad de cada
archivo. Regla: la UI
   (components/) nunca importa Supabase; solo habla con el hook.
2. Escribe los tipos de dominio en src/types/emissions.ts.
3. Escribe una primera versión de EmissionsTracker.tsx con datos de ejemplo
en estado local […]
5. Lista los estados de la interfaz que el hook tendrá que exponer en el
siguiente paso.
```

Prompt completo: `bitacora/01-prompt-estructura.md`. Respuesta: `bitacora/v1/respuesta.md`.

Qué devolvió (v1):

- Árbol por capas (`types → services → hooks → components`).
- Tipos sin `enum`, porque `erasableSyntaxOnly` no los permite: `EMISSION_CATEGORIES` como tupla `as const`, de la que sale el tipo.
- El contrato `UseEmissionsResult`, escrito antes que el hook.
- Un componente de 315 líneas con `useState` y cinco mediciones de ejemplo.

Qué se vio al verificar: compila y pasa el lint, pero los datos siguen en el componente, el total se recalcula en cada tecla, `replace(',', '.')` cambia solo la primera coma (así que `1.234,5` da `NaN`) y el error de categoría se calcula pero nunca se muestra.

---

2. Prompt de lógica: el hook useEmissions

Qué se buscaba: sacar los datos de la UI. Se pidieron el cliente de Supabase que no falla sin credenciales (modo demo), un servicio que valide cada fila en vez de usar `as`, inserción pesimista, errores de carga e inserción separados, protección contra el doble montaje de StrictMode, el repositorio inyectable para probar sin red, `schema.sql` con RLS y cuatro pruebas.

```
1. src/lib/supabaseClient.ts: crea el cliente SOLO si existen
VITE_SUPABASE_URL y
   VITE_SUPABASE_ANON_KEY. Si falta alguna, exporta null en vez de lanzar
un error […]
2. src/services/emissionsService.ts: […] valida cada fila que llega: si la
categoría no es
   una de las tres, descarta la fila en vez de hacer un cast con "as".
   Ojo: numeric de Postgres puede llegar como string por PostgREST.
4. src/hooks/useEmissions.ts que implemente UseEmissionsResult: […] El hook
recibe el
   repositorio por parámetro (con el real o el demo por defecto) para poder
probarlo con
   Vitest sin red.
6. supabase/schema.sql con la tabla, un check de categoría y de co2_kg >=
0, y RLS.
7. Pruebas de Vitest para el hook con un repositorio falso […]
```

Prompt completo: `bitacora/02-prompt-logica-hook.md`. Diferencias con v1: `bitacora/v2/cambios-desde-v1.diff` (10 archivos, +580 −277).

El cambio central, en `EmissionsTracker.tsx`:

```
-  // Paso 1: estado local. Paso 2: const { emissions, ... } =
useEmissions()
-  const [emissions, setEmissions] = useState<readonly
Emission[]>(SAMPLE_EMISSIONS)
+  const { mode, emissions, status, loadError, isSubmitting, submitError,
addEmission, reload, clearSubmitError } =
+    useEmissions()
```

Qué se vio al verificar (`v2/verificacion.txt`):

- **2 de 4 pruebas fallan por timeout.** La prueba crea el repositorio dentro de `renderHook(() => useEmissions(fakeRepository()))`, así que en cada render llega uno nuevo; como `reload` depende de él, el efecto vuelve a cargar sin fin. La prueba no está mal: muestra un hook frágil.
- oxlint marca dos avisos: `set-state-in-effect` y `only-export-components`.
- **RLS, probado como `anon` en PostgreSQL 17 (`v2/rls-pruebas.txt`): 5 de 9 pruebas fallan.** El cliente puede fijar `measured_at` y elegir el `id`, `update` y `delete` no dan error, y **`truncate` vació la tabla**. RLS habilitado con dos políticas no basta: Supabase da todos los privilegios a `anon` en cada tabla nueva, y RLS no filtra `truncate`.

- Bundle: 228,41 kB sin variables y 444,55 kB con variables de prueba. La diferencia es Supabase, que Vite elimina como código muerto si no hay credenciales.

---

3. Prompt de refinamiento: TypeScript estricto, accesibilidad y useMemo

Qué se buscaba: las tres restricciones del enunciado (accesibilidad, TypeScript estricto y `useMemo`), más la corrección de cada fallo visto en v2. Cada exigencia es verificable:

- banderas concretas del compilador;
- «ningún `as` salvo `as const`»;
- una lista de entradas que el parser debe aceptar y rechazar;
- roles ARIA concretos;
- `memo` y `useMemo` en lugares concretos.

```
A. TypeScript estricto de verdad
   - Activa "noUncheckedIndexedAccess", "exactOptionalPropertyTypes" y
"noImplicitOverride" […]
   - Ningún "as" salvo "as const". parseEmissionRow tiene que estrechar el
tipo con
     comprobaciones, no con "row as Record<string, unknown>".
B. Bugs encontrados en la verificación
   - 2 pruebas fallan por timeout: el hook recarga sin fin cuando el
repositorio no es estable
     entre renders. El hook debe ser robusto a eso sin pedirle nada a quien
lo llama. […]
   - Debe aceptar "1.234,5" (formato colombiano), "1234.5" y "1234,5";
rechazar "1,2,3", "abc",
     "-3", "Infinity" y el vacío.
   - Evita doble envío […]. Agrega la prueba.
C. Accesibilidad (WCAG 2.1 AA): role="status"/aria-live, role="alert", foco
al primer campo
   con error, error de categoría visible, total legible para lectores de
pantalla […]
D. Re-renders: useMemo para el total y el desglose; React.memo en
EmissionCard; el objeto
   que devuelve el hook memorizado con useMemo […]
```

Prompt completo: `bitacora/03-prompt-refinamiento.md`. Diferencias con v2: `bitacora/v3/cambios-desde-v2.diff` (13 archivos, +541 −239).

La corrección del bug de v2, en `useEmissions.ts`:

```
 export function useEmissions(repository: EmissionsRepository =
defaultRepository): UseEmissionsResult {
+  const [repo] = useState(() => repository)
+
   const [emissions, setEmissions] = useState<readonly Emission[]>([])
-  const [status, setStatus] = useState<LoadStatus>('idle')
+  // Empieza en 'loading': así el efecto no necesita un setState síncrono.
+  const [status, setStatus] = useState<LoadStatus>('loading')
```

Errores atrapados por `exactOptionalPropertyTypes`: v2 borraba un error con `setErrors((prev) => ({ ...prev, [field]: undefined }))`. Con la bandera nueva eso ya no compila, y v3 quita la clave con `withoutError()`. Lo mismo pasa con `aria-describedby={cond ? id : undefined}`, que v3 resuelve con props condicionales.

Re-renders: el resumen pasa a `useMemo(() => summarizeEmissions(emissions), [emissions])`, y `EmissionCard` y `EmissionsSummary` se envuelven en `memo`. Hay una prueba que comprueba que el objeto que devuelve el hook es el mismo tras un `rerender` sin cambios.

Qué se vio al verificar (`v3/verificacion.txt`): `tsc` sin errores con las banderas nuevas y **27 de 27 pruebas verdes**: 7 del hook, entre ellas la regresión del repositorio inestable y el doble envío, y 20 de validación. Pero **oxlint pasa de 2 avisos a 1 error y 2 avisos**:

- `exhaustive-deps`: se lee un ref en el cleanup.
- `set-state-in-effect`: sigue ahí. Separar `fetchList` de `reload` no bastó.
- `prefer-tag-over-role`: aparece por el `role="status"` nuevo.

El refinamiento arregló lo que se pidió y dejó a la vista lo que el linter todavía no aceptaba.

---

4. Prompt de auditoría: lint en cero, RLS y estados verificables

Qué se buscaba: cerrar los cuatro ejes de la Auditoría IA del módulo. Se le dio a la IA la salida exacta de oxlint y de las pruebas de RLS, con instrucciones de arquitectura concretas: la carga como suscripción con bandera local, `<output>` en vez de `role="status"`, privilegios por columna y un `?simular=` que solo actúa en modo demo.

```
1. Lint en cero (errores y avisos):
   - No leas ref.current en el cleanup del efecto.
   - Ningún setState alcanzable de forma síncrona desde el efecto de carga.
Escribe la carga
     inicial como una suscripción: el efecto lanza la promesa, guarda una
bandera local
     "activo" y solo aplica el resultado en el .then si sigue activo. […]
   - Haz cumplir la arquitectura en .oxlintrc.json con no-restricted-
imports […]
2. RLS: revoke all a anon y authenticated; grant select, y grant insert
SOLO sobre las
   columnas source_name, category y co2_kg. Sin update, delete ni truncate.
[…]
   Debe pasar las 9 pruebas de supabase/pruebas/rls.sql.
3. Estados verificables en modo demo: ?simular=lento | error-carga | error-
guardado | vacio […]
```

Prompt completo: `bitacora/04-prompt-auditoria.md`. Diferencias con v3: `bitacora/v4/cambios-desde-v3.diff` (9 archivos, +153 −49).

La carga como suscripción (`useEmissions.ts`):

```
useEffect(() => {
  let active = true
  repo.list().then(
    (rows) => { if (!active) return; setEmissions(rows);
setStatus('success') },
    (error: unknown) => { if (!active) return;
setLoadError(messageOf(error)); setStatus('error') },
  )
  return () => { active = false }
}, [repo, loadRequest])
```

Privilegios antes que políticas (`supabase/schema.sql`):

```
revoke all on table public.emissions from public, anon, authenticated;
grant select on table public.emissions to anon, authenticated;
grant insert (source_name, category, co2_kg) on table public.emissions to
anon, authenticated;
```

Qué se vio al verificar: `tsc` sin errores, **oxlint en 0**, **37 de 37 pruebas** (10 nuevas del modo demo) y **RLS 9 de 9**. El schema se aplicó dos veces sin error; el de v2 fallaba la segunda vez. La regla de arquitectura se probó con archivos que tenían imports prohibidos, y oxlint dio los 4 errores esperados (`v4/lint-regla-arquitectura.txt`).

---

5. Ajuste final tras revisar las capturas

La captura de carga lenta mostró algo que ninguna herramienta había detectado: mientras carga, el resumen decía **«0 kg CO_2e»**, como si la empresa no emitiera nada. Se cambió a mano para que muestre «—» con `aria-busy` mientras carga, y se agregó `motion-reduce:animate-none` al esqueleto y al spinner. Diferencias: `bitacora/ajuste-final-tras-capturas.diff`. Esta es la versión que queda en `src/`.

---

6. Auditoría IA

6.1 Análisis de código

Verificación	Resultado	Evidencia
`pnpm typecheck` (`tsc -b --noEmit`) con `strict`, `noUncheckedIndexedAccess` y `exactOptionalPropertyTypes`	0 errores	`docs/evidencia/verificacion-final.txt`
`pnpm lint` (oxlint con react, typescript y jsx-a11y)	0 errores, 0 avisos	ídem
La regla de arquitectura (`no-restricted-imports`) detecta imports prohibidos	4 de 4 detectados	`bitacora/v4/lint-regla-arquitectura.txt`
`pnpm test` (Vitest, happy-dom)	37 de 37	ídem
Casts con `as` en `src/`, sin contar `as const` ni las pruebas	0 (el único `as` que aparece es un alias de import)	`grep` sobre `src/`

Hallazgo de la auditoría: **`pnpm tsc --noEmit` no revisa nada en este proyecto**. El `tsconfig.json` raíz tiene `"files": []` y solo referencias, así que el comando sale con 0 incluso con un error de tipos a propósito. `tsc -b` sí lo detecta (`docs/evidencia/tsc-noemit-no-revisa-nada.txt`). Por eso el chequeo real es el script `typecheck`.

6.2 Optimización de prompts

Prompt	Lo que lo volvió específico
1	Versiones exactas y opciones del compilador; prohibido proponer otro stack; solo arquitectura
2	Contrato del paso 1 como entrada; cada requisito del hook se puede comprobar (inserción pesimista, errores separados, StrictMode, inyección)
3	Salida real de las herramientas como contexto; casos de entrada concretos; banderas concretas del compilador
4	Errores exactos de oxlint y de RLS; la solución de arquitectura indicada en una frase; criterio de aceptación numérico (9 de 9, lint en 0)

Lo que no funcionó: en el prompt 3, separar `fetchList` de `reload` no eliminó `set-state-in-effect`. Pedir «quita el aviso» no bastó: hubo que describir el patrón (una suscripción con bandera local) en el prompt 4.

6.3 Tamaño del bundle

Compilación	JS	JS gzip	CSS	CSS gzip
Sin variables de entorno (modo demo)	233,20 kB	73,15 kB	18,53 kB	4,63 kB
Con variables de prueba (valores falsos)	**449,37 kB**	**128,33 kB**	18,53 kB	4,63 kB

El segundo es el tamaño que verá el usuario con Supabase. Sin credenciales, Vite reemplaza `import.meta.env.VITE_*` por `undefined` y elimina `@supabase/supabase-js` como código muerto. Si solo se midiera el primero, el tamaño quedaría subestimado en unos 216 kB.

6.4 Verificación RLS

Prueba: `bash herramientas/probar-rls.sh`. Crea un PostgreSQL 17 desechable en Docker con un stub de los roles y privilegios por defecto de Supabase, aplica el schema dos veces y corre 9 pruebas como `anon`.

#	Prueba	v2	v4
1	anon lee	OK	OK
2	anon inserta una medición válida	OK	OK
3	Categoría fuera de la lista	OK (check)	OK (política)
4	`co2_kg` negativo	OK (check)	OK (política)
5	El cliente fija `measured_at`	**FALLA**	OK
6	El cliente elige el `id`	**FALLA**	OK
7	`update`	**FALLA** (sin error)	OK (denegado)
8	`delete`	**FALLA** (sin error)	OK (denegado)
9	`truncate`	**FALLA** (vació la tabla)	OK (denegado)

Límite: el stub reproduce los roles y privilegios por defecto de Supabase, pero no es Supabase. `schema.sql` no se ha ejecutado en un proyecto real porque no hay credenciales.

---

7. Capturas

Todas están en `docs/capturas/`, en modo demo, servidas con `vite preview` y sacadas en Chrome a 1366×900 (escritorio) y 390×844 (móvil).

Archivo	Qué muestra
`01-escritorio-lista.png`	Resumen por categoría y las cinco cards
`02-escritorio-validacion.png`	Errores por campo y foco en el primero
`03-escritorio-guardando.png`	Botón «Guardando…» deshabilitado
`04-escritorio-medicion-agregada.png`	«Caldera planta norte» (1.234,5 kg) primera en la lista; total en 10,13 t
`05-carga-esqueleto.png`	`?simular=lento`: esqueleto y resumen en «—»
`06-error-de-carga.png`	`?simular=error-carga`: mensaje y «Reintentar»
`07-lista-vacia.png`	`?simular=vacio`
`08-error-al-guardar.png`	`?simular=error-guardado`: mensaje de RLS y lo escrito se conserva
`09-movil-inicio.png`, `10-movil-cards.png`, `11-movil-validacion.png`	Móvil a una columna, sin scroll horizontal (`scrollWidth` 390 = `innerWidth` 390)

---

8. Qué es real y qué es simulado

Pieza	Estado
Componente, hook, validación y pruebas	Ejecutado: 37 pruebas, build y capturas
Modo demo	Real y marcado en pantalla. Datos de ejemplo, no mediciones de una empresa
Cliente y servicio de Supabase	Compilados y con los tipos revisados. **No ejecutados contra un proyecto real** (no hay credenciales)
`schema.sql` y RLS	Ejecutado en PostgreSQL 17 con un stub de Supabase. No ejecutado en Supabase
Las cifras de CO_2	De ejemplo. No salen de factores de emisión